

Lucas' Sernik Peer Review of Akash's Goda Second Draft

Akash investigates the following underexplored question: Does AI actually improve programmers capacity to code in professional settings? The author brings a point that has yet not been greatly investigated in literature. While there are papers that evaluate the increase in commit output and faster task completion when programmers are actively assisted by AI, many of them only evaluate raw quantity, making no claim supporting that the code written with the help of AI is also good quality.

In order to validate the quality of a piece of code, the author is proposing to analyze churn rate. He defines churn rate as the rate of modified files in a repository that are re-modified within a relatively short period of time over total files modified in a given month. His theory is that, if a certain file is modified within a short period after a commit, it is likely that the commit contained code that was generated by AI which was initially committed, proved itself to be buggy, and then rewritten by programmers. He would like to explore this gap and argue that AI might have made programmers slower, indicating less or even no true gain in productivity with the adoption of AI.

He also wants to relate such recalculated change in productivity to Acemoglu's estimate of TFP growth provided by AI, who used data from papers that evaluated quantity instead of quality.

Finally, he wishes to analyze the impact of these AI tools in different subindustries within the greater IT industry.

I believe that the paper contains solid literature background and contains a relevant question: what should be the metric to analyze whether AI is making programmers more productive or not. In my view, this is the greatest contribution that the paper could make. One of the issues that I find when the author proposes to change TFP projections based on his findings. Because he is using data from 2020-2022, his estimates of productivity wouldn't be as relevant, since the models have evolved greatly since then and the data he is using is somehow "outdated". He is also only considering the use of AI through Copilot, which might not be reasonable if he is trying to make such estimate. However, if he frames his paper as the proposition of an alternative metric to measure AI productivity gains, he could have great impact.

Secondly, he could better identify the scope of projects that were in GitHub at the given timer period. I am not sure about this, but, since he has data on public repositories, it would be interesting to see how does a public repository differ from a private repository, both in terms of content/projects and of AI use.